MICAI 2026

TUTORIAL · MICAI 2026

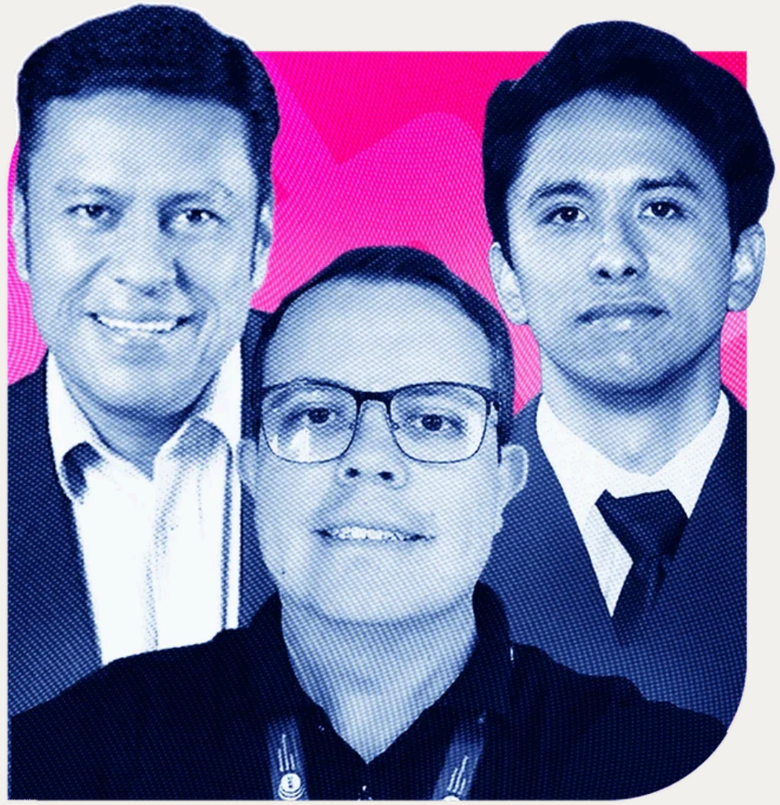
Opening the Black Box

A Hands-On Introduction to Mechanistic Interpretability of Large Language Models

Tec de Monterrey, Campus Chihuahua · Friday 6 November 2026

Miguel González · **Salvador Hinojosa** · **Víctor Terrón-Macías**

Tecnológico de Monterrey



INTRODUCTION

In three hours you will read a model's internals — and know the limits

1

What it is

What mechanistic interpretability is, and what it can — and cannot — claim.

2

Why it matters

Implications for safety, for auditing AI systems, and for science.

3

How to use it

Transfer one method to your own research, with an honest sense of what it proves.

Format: every experiment starts with a prediction, then you run it in Colab.

INTRODUCTION

Five blocks: observe, intervene, decompose, trace, audit

TIME	BLOCK	WHAT WE DO	PRESENTER
0:00	**Hook**	Why look inside? Live steering demo	Miguel González
0:08	**1 · Observe**	Residual stream, logit lens · NB1	Miguel González
0:30	**2 · Intervene**	Causality, activation patching · NB2	Salvador Hinojosa
1:00	**Break**	Switch Colab to a T4 GPU	—
1:10	**3 · Decompose**	Superposition, SAEs, steering · NB3	Salvador Hinojosa
1:40	**4 · Trace**	Attribution graphs, circuits · NB4	Salvador Hinojosa
2:10	**5 · Audit**	Can we trust an LLM judge? · NB5	Víctor Terrón-Macías
2:50	**Wrap**	Implications, open problems, resources	Víctor Terrón-Macías

SETUP · RUNS WHILE WE TALK

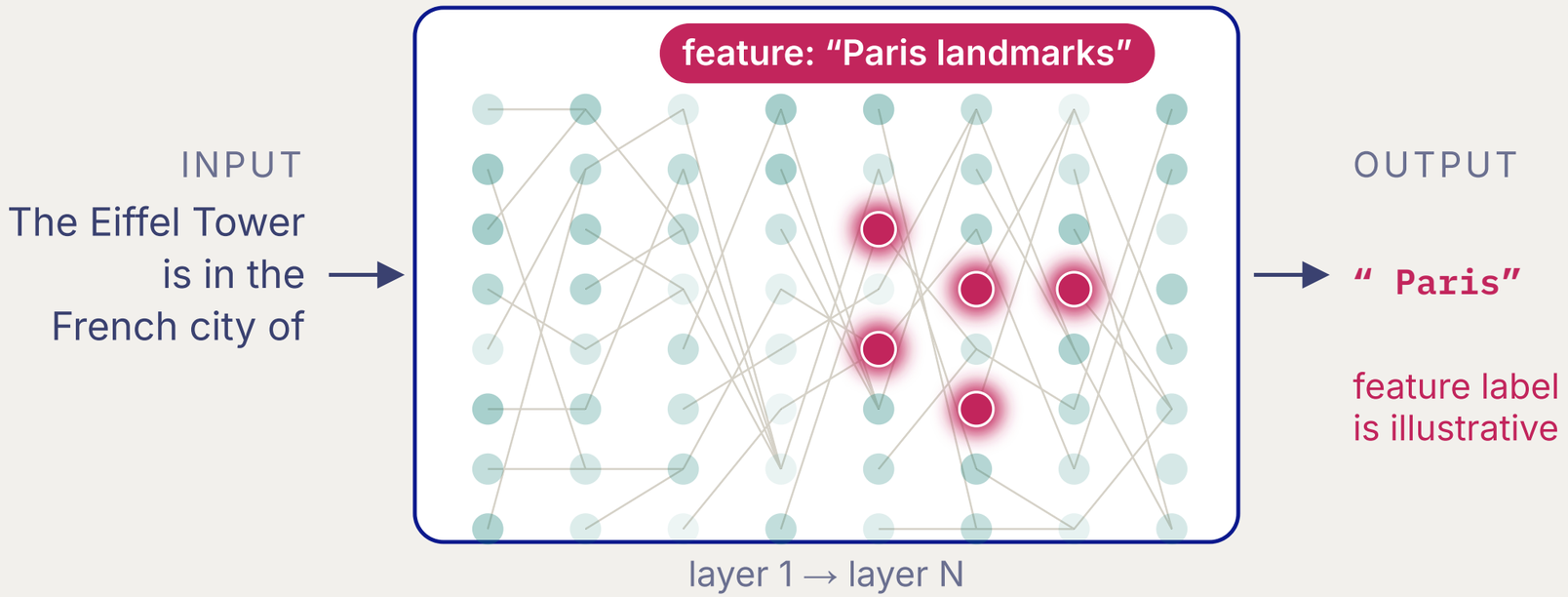
Open NB0 now — it installs while we talk

1. Open NB0 in Colab from **`salvahin.github.io/mechinterp-tutorial`**
2. Add your Hugging Face token as Colab secret `HF_TOKEN`
3. Accept the licence of `google/gemma-3-1b-it` on Hugging Face
4. Runtime → `Run all`. Look for the green check.

Stuck?

Raise your hand — our assistant will come to you. Keep listening; nothing in the next 8 minutes needs a laptop.

Tests see outputs; interpretability reads the computation



Testing: inputs in, outputs out.

Interpretability: read the steps in between.

SHOW OF HANDS

Where does GPT-2 “know” that the Eiffel Tower is in Paris?

A In the input embedding of “Eiffel”

B Spread evenly across all layers

C In specific middle-to-late layers

D Nowhere — it is just statistics

Remember your answer. You will check it yourself in NB1.

WHY NOW

The tools are open now; the methods are still young

RECOGNISED

A 2026 breakthrough

MIT Technology Review lists mechanistic interpretability among its 10 Breakthrough Technologies of 2026.

OPEN

Runs on free Colab

TransformerLens, SAELens, Neuronpedia, circuit-tracer, Gemma Scope 2 — everything we use today.

YOUNG

Many open problems

Methods lack ground truth and validation standards; results often fail to generalise.

MIT Technology Review, “10 Breakthrough Technologies 2026”. Gemma Scope 2: Google DeepMind (2025). Sharkey et al., “Open Problems in Mechanistic Interpretability”, TMLR 2025.

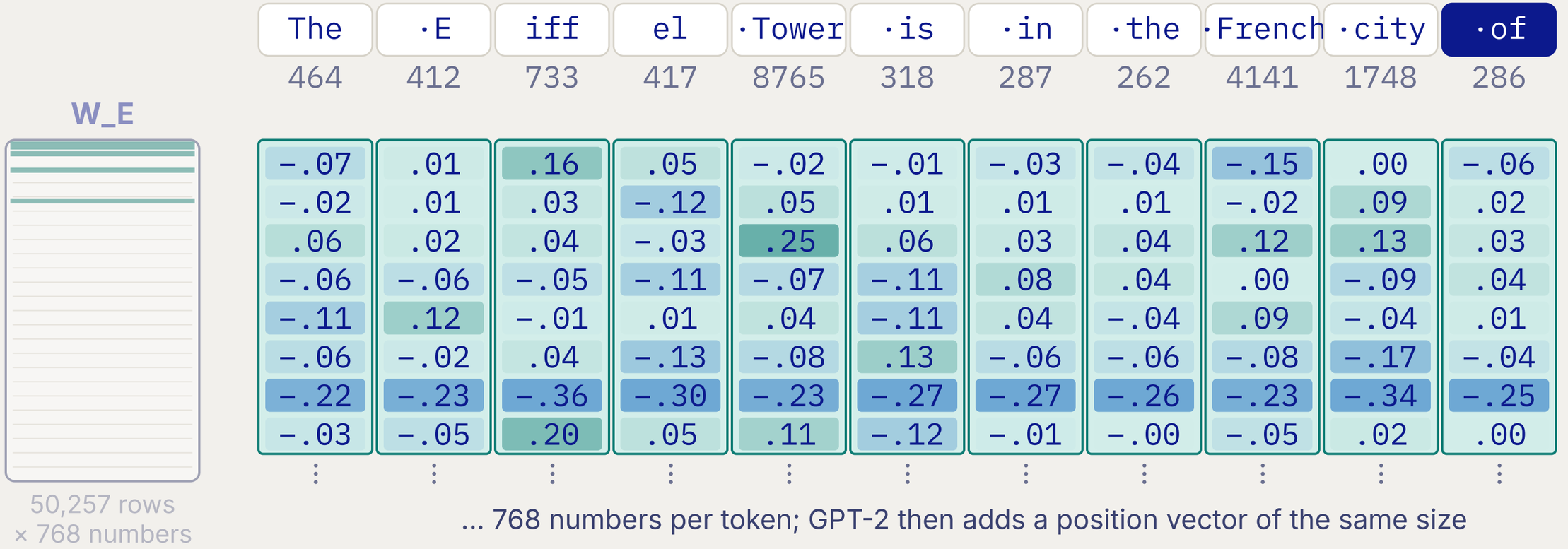
BLOCK 1 · 0:08–0:30

1

Observe

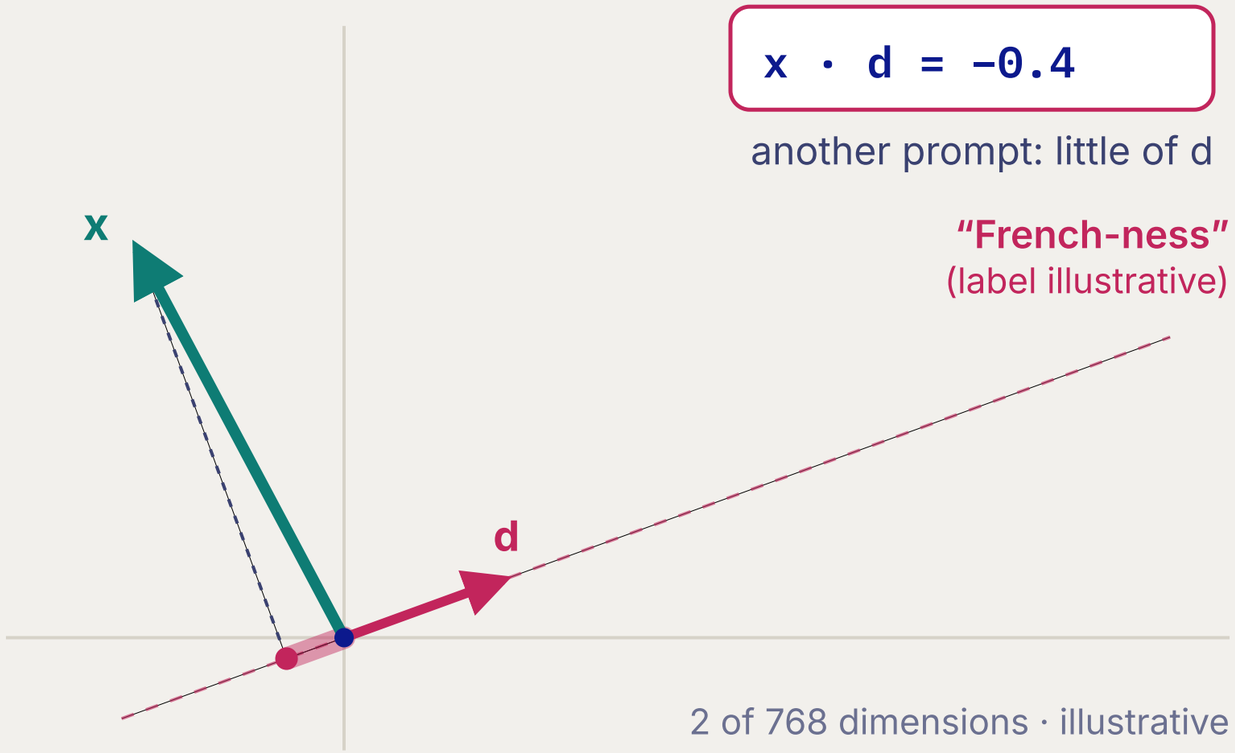
What is inside a transformer, and can we read it as it computes?

Text becomes tokens, tokens become vectors



That row is the token's **vector**: 768 numbers (first 8 shown, real GPT-2 values).

A direction in activation space can mean something



$x \cdot d = -0.4$

another prompt: little of d

“French-ness”
(label illustrative)

2 of 768 dimensions · illustrative

The same move, all day:

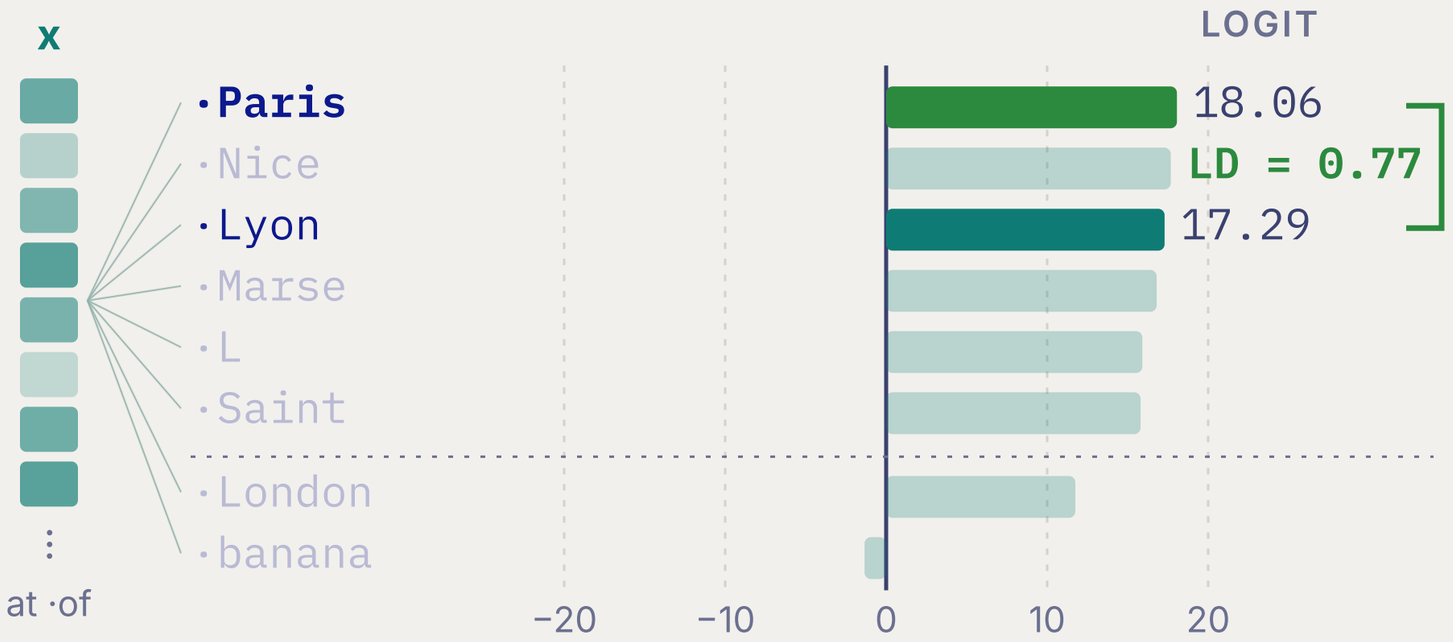
Logit lens — Block 1
`x · (a token's vector)`

Probe — Blocks 3, 5
`x · w, w learned`

SAE feature — Blocks 3, 4
$x \cdot w_{enc}$, then (Jump)ReLU

Steering — Block 3
`x + α·d: writes along d`

Logits are scores; softmax turns them into probabilities

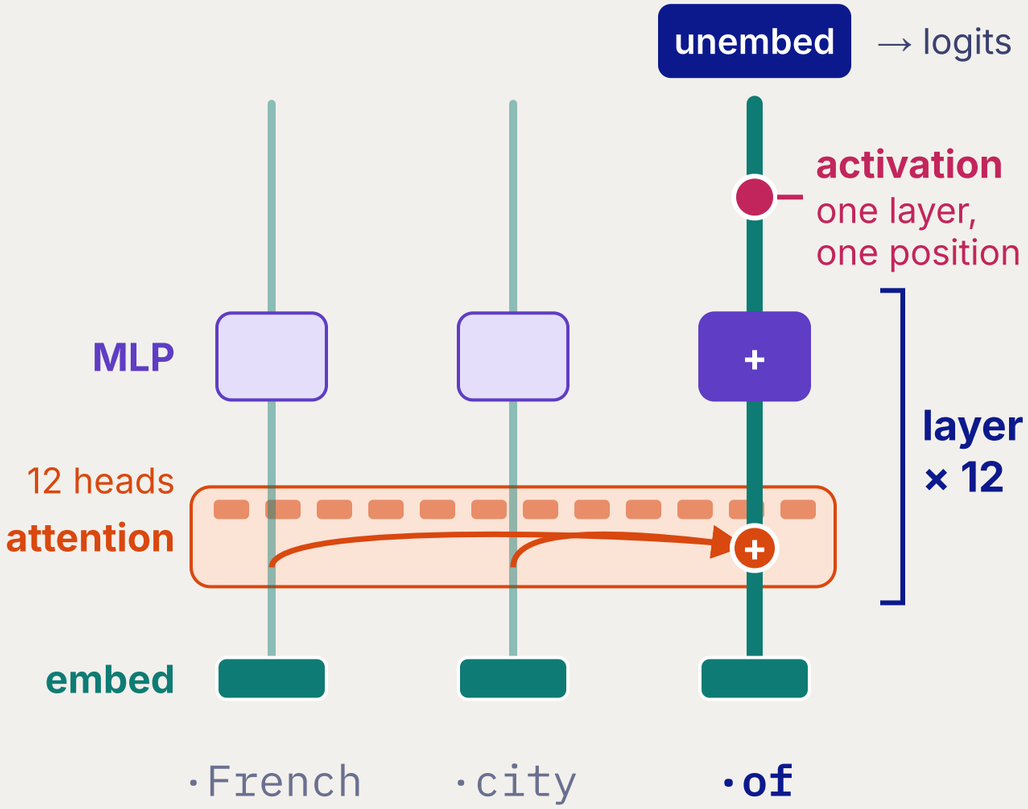


$$LD = \text{logit}_{\text{Paris}} - \text{logit}_{\text{Lyon}} = 18.06 - 17.29 = 0.77$$

Linear in x, unlike probabilities. Block 2 measures with LD.

Real values: GPT-2 small, TransformerLens (logits centred to mean 0 over the vocabulary), scratch/primer/primer_numbers.py.

A transformer is layers that add to a running sum



Residual stream
the running sum: 768 numbers per position

Attention
moves information between positions (12 heads)

MLP
transforms one position at a time

Layer
one attention + one MLP; GPT-2 small stacks 12

Activation
the numbers at one layer and one position

Three things we do to activations today

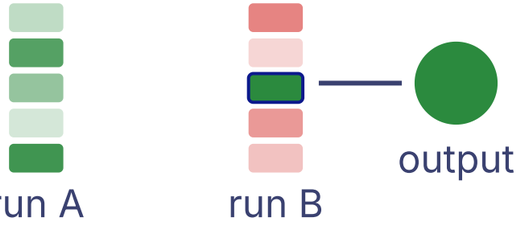
READ



Decode what an activation contains.

- Block 1 · logit lens
- Block 3 · probes
- Block 5 · probes on a judge

EDIT



Change it, then watch the output.

- Block 2 · patching
- Block 3 · steering

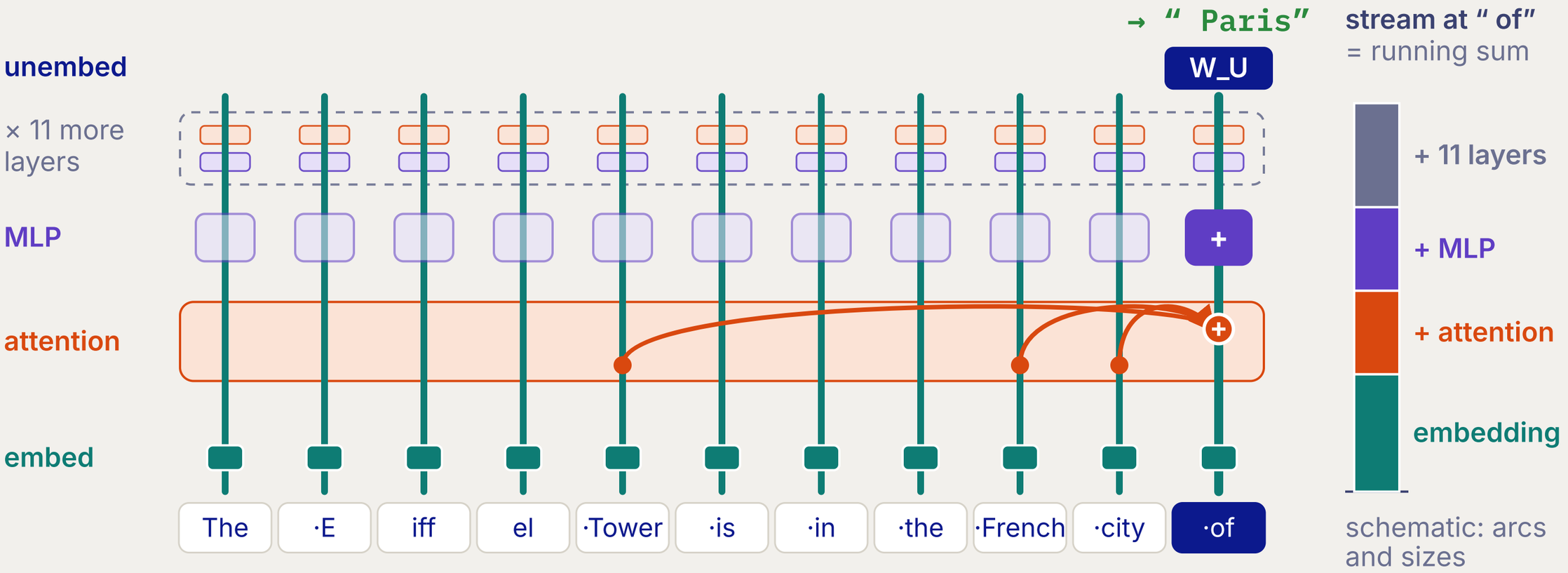
DECOMPOSE



Split it into interpretable parts.

- Block 3 · SAEs
- Block 4 · attribution graphs

Every layer reads from one shared stream and adds to it



After 12 layers, **unembedding** reads the final stream at “ of” → next token “ Paris”.

LIVE DEMO · 1 MINUTE

A real GPT-2 runs in your browser — watch it predict

poloclub.github.io/transformer-explainer

The stream

Embedding, then 12 blocks stacked on the same vectors.

Attention

Click a head: which earlier tokens “of” looks at.

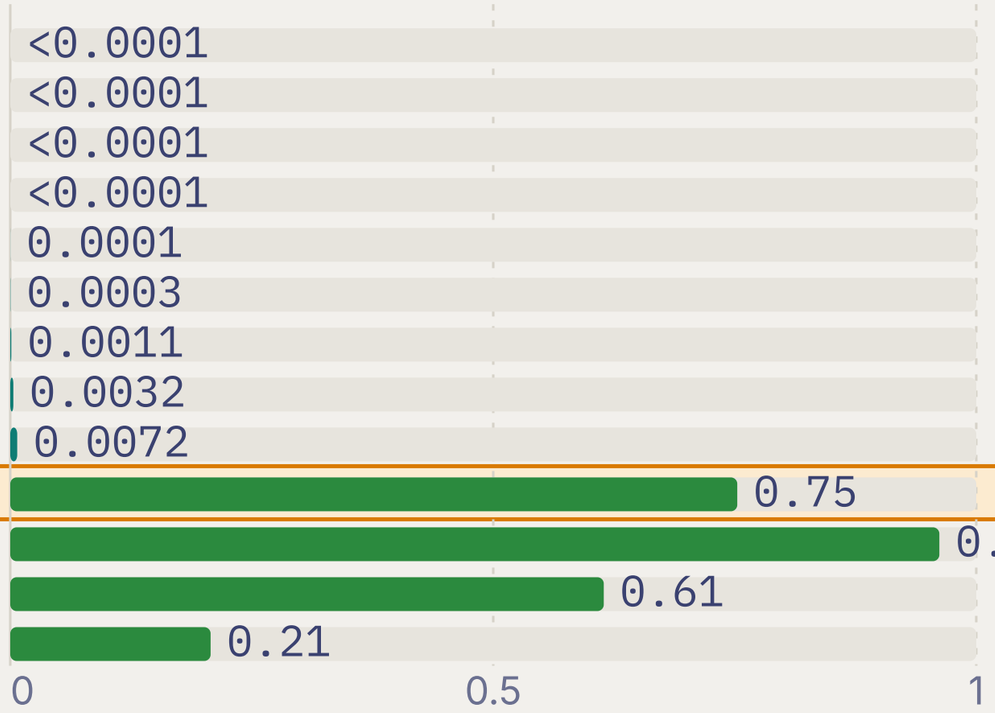
Prediction

Next-token probabilities; move the temperature slider.

Transformer Explainer: Cho et al., Georgia Tech Polo Club (2024). Runs GPT-2 small client-side.

Decoding every layer shows “Paris” emerging late

RESIDUAL AFTER	TOP-1 TOKEN	P(·PARIS)	RANK OF ·PARIS
embed	·destro	<0.0001	23,918
block 0	·the	<0.0001	5,323
block 1	·the	<0.0001	4,093
block 2	·the	<0.0001	1,804
block 3	·the	0.0001	374
block 4	·the	0.0003	233
block 5	·the	0.0011	128
block 6	·La	0.0032	46
block 7	·Man	0.0072	21
block 8	·Paris	0.75	1
block 9	·Paris	0.96	1
block 10	·Paris	0.61	1
block 11	·Paris	0.21	1



“ Paris” is top-1 from **block 8** on (after 9 of 12 blocks). Real GPT-2 small values.

BEFORE NB1

Predict first: remove one word

In NB1 you run the logit lens yourself, on the prompt **without** “French”:

```
The Eiffel Tower is in the city of
```

PREDICT Before running: at which block does “ Paris” first become the top prediction — if at all? Write it down. _____

HANDS-ON · 0:18–0:28

NB1: run the logit lens yourself (10 minutes)

1. **Run** the logit lens on the Eiffel prompt. Check your prediction.
2. **Your turn:** write the logit lens yourself, 3 lines. Then try your own prompt.
3. **Stretch:** find induction heads with an induction score.

Notice

Which prompts resolve early, late, or never? Note one surprise to share.

CPU runtime is fine — GPT-2 small loads in under a minute.

HONEST LIMITS

Observation shows correlation, not cause

The logit lens can tell us

By block 8, the stream already points toward “ Paris”.

It cannot tell us

That block 8 *causes* the answer, or which component computes it.

- Attention patterns show where a head looks, not what it moves or whether it matters.
- Early layers decode poorly: their basis drifts from the output's. The tuned lens learns a translator per layer.

nostalgebraist (2020) “interpreting GPT: the logit lens”. Belrose et al. (2023) “Eliciting Latent Predictions from Transformers with the Tuned Lens”. Jain & Wallace (2019) “Attention is not Explanation”.

HANDOFF

We can see the answer forming — but does that layer cause it?

- Next: Salvador Hinojosa · Block 2 · Intervene

0:30 · SALVADOR HINOJOSA

2

Intervene: from seeing to causing

The logit lens showed the answer forming. Now we change the inside and watch the output.

BLOCK 2 · INTERVENE

The next 100 minutes are one story: cause, units, circuits

BLOCK 2 · 0:30

Cause

Which activations does the model actually rely on?

`activation patching`

BLOCK 3 · 1:10

Units

What are the variables inside those activations?

`SAEs · steering · probes`

BLOCK 4 · 1:40

Circuits

How do those units combine into one answer?

`attribution graphs`

Each act answers the question the previous one leaves open.

BLOCK 2 · INTERVENE

Seeing a representation does not show that the model uses it

OBSERVE

Read an activation

Logit lens, probes: the information is **present**. A decoder can find it even if no later layer ever reads it.

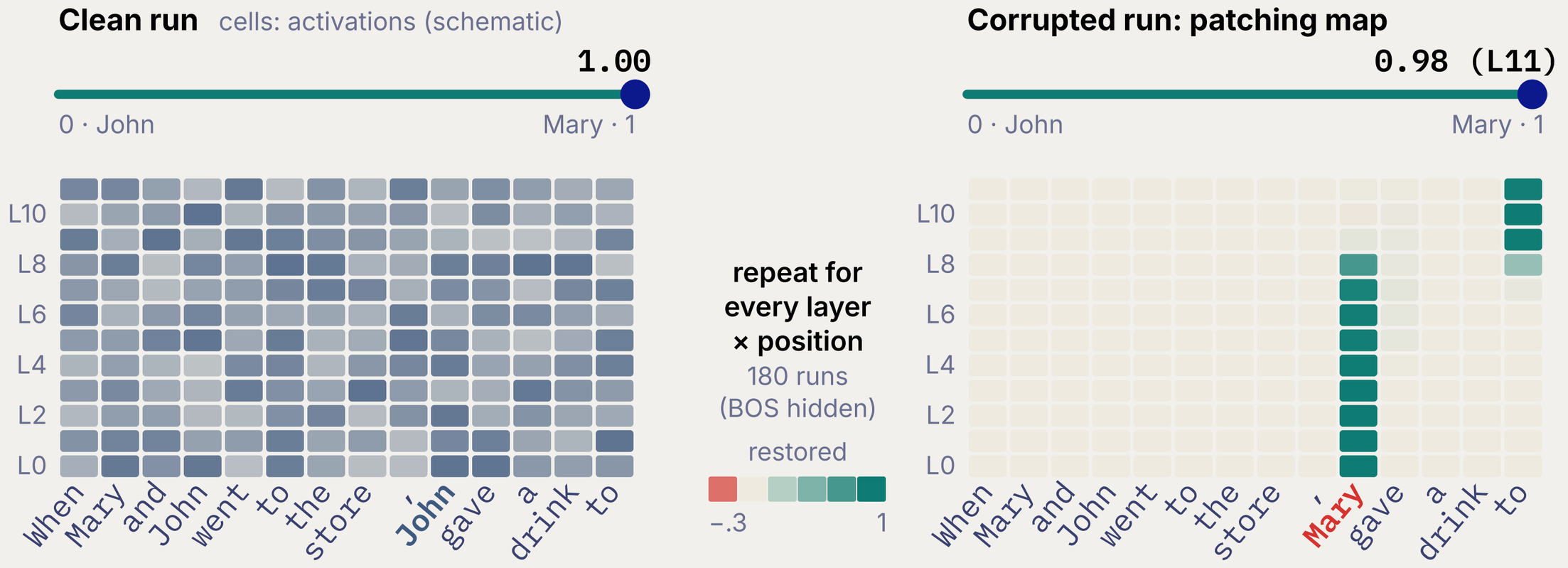
INTERVENE

Change an activation, measure the output

If the output moves, the model **uses** that activation. This is a causal claim about the network.

> Does that layer cause it? Only an intervention can say.

A patch transplants one activation between runs



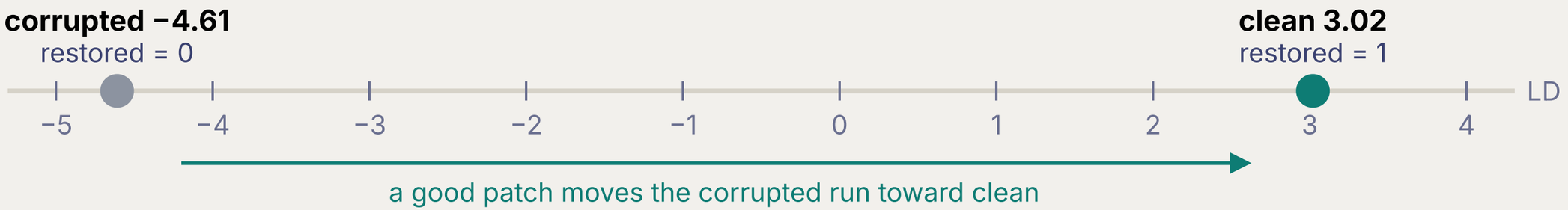
BLOCK 2 · INTERVENE · THE METRIC

We score each patch by how far it restores the clean answer

$$\mathrm{LD} = \mathrm{logit}(\mathrm{IO}) - \mathrm{logit}(\mathrm{S})$$

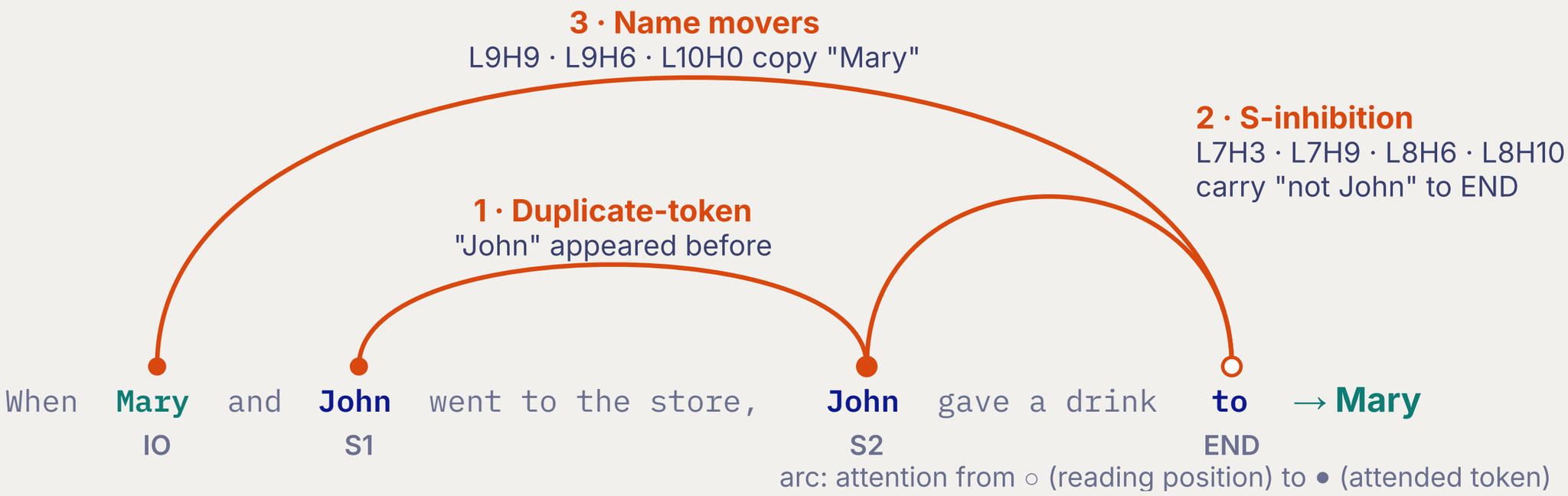
IO = indirect object, the right answer (Mary) · **S** = subject, the repeated name (John)

$$\mathrm{restored} = (\mathrm{LD}_{\mathrm{patched}} - \mathrm{LD}_{\mathrm{corr}}) \,/\, (\mathrm{LD}_{\mathrm{clean}} - \mathrm{LD}_{\mathrm{corr}})$$



Computed on GPT-2 small (TransformerLens), mean over NB2's 8 prompts: $\mathrm{LD}_{\mathrm{clean}} = +3.02$, $\mathrm{LD}_{\mathrm{corr}} = -4.61$.

Indirect object identification is the canonical worked circuit



Wang et al. 2022, “Interpretability in the Wild”, GPT-2 small. Simplified: the paper also reports backup and negative name movers.

BLOCK 2 · INTERVENE

Before you run it: where will the patch matter?

PREDICT

Which layers and positions will matter most when you patch the residual stream?

Sketch a grid of 12 layers × 14 tokens on paper. Shade the cells you expect to restore Mary.

Positions

First name? Second John? Final token? All of them?

Layers

Early, middle, late, or a band that moves?

Commit

Write it in the notebook cell before running.

NB2 · PATCHING ≈ 14 min · GPT-2 small · CPU is fine

Find where IOI happens, then find the heads

1. Check the 8 clean and corrupted prompts and the metric (given).
2. Write the patching hook.
3. Patch the residual stream at every layer × position.
4. Patch each attention head; rank them.
5. Does attention predict importance?

Done when

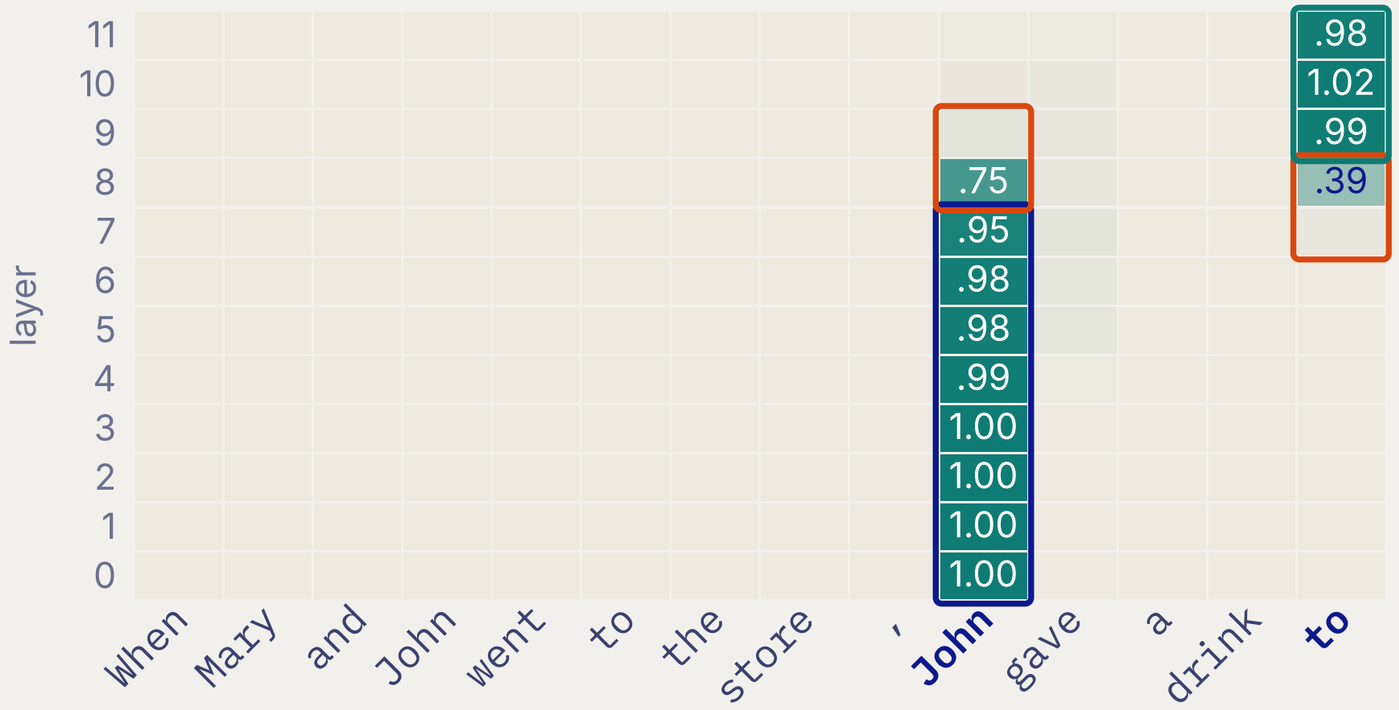
Your heatmap shows the handover from S2 to the final token, and you can name the heads that carry it.

Stuck?

Raise a hand: the assistant is on Colab duty.

BLOCK 2 · INTERVENE · OUR RESULT

Information moves from the repeated name to the final token in layers 7–8



restored (0 = corrupted, 1 = clean)

−.3 1

1 Early: all at S2
the only token that differs
restores ≈ 1 up to L7

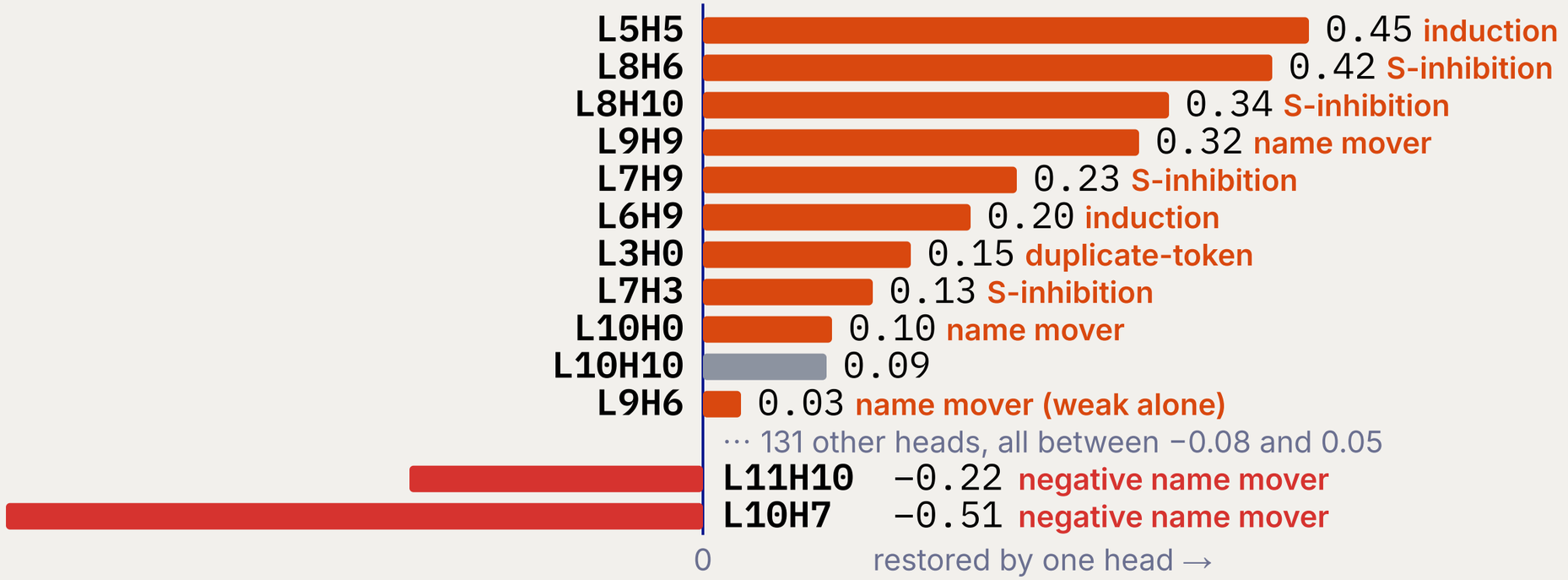
2 Handover in L7–8
S2: 0.95 → 0.75 → 0.05
END: 0.03 → 0.39 → 0.99

3 Late: all at the final token
≈ 1 from layer 9 on
earlier tokens: 0 (identical)

GPT-2 small · resid_pre
mean over NB2's 8 prompts

BLOCK 2 · INTERVENE · HEAD-LEVEL PATCHING

Head patching finds S-inhibition and name-mover heads, and two that fight back



Computed: GPT-2 small, NB2's 8 prompts, each head's output patched at all positions. Role labels from Wang et al. 2022. NB2 §5: attention to IO vs causal effect over 144 heads, r = −0.16.

Patching tells us where, not what

We now know

Which layer, position and head the answer depends on, for this task, measured causally.

We still do not know

What is written in that 768-dimensional vector. "Name info at layer 9" is a location, not a variable.

Next: what are the units inside an activation?

1:00 – 1:10

Break · 10 min

Before you leave your seat, prepare Block 3:

1. Colab: Runtime → Change runtime type → **T4 GPU**.
2. Open **NB3** and run the first cells to preload Gemma weights.
3. No GPU or an error? The student assistant will help.

Optional, Block 4 on your own GPU: NB4 needs its own fresh runtime and free Colab gives one T4 per account. Start its 18 GB download only if you will skip NB3 on GPU, or in a second account.

1:10 · SALVADOR HINOJOSA

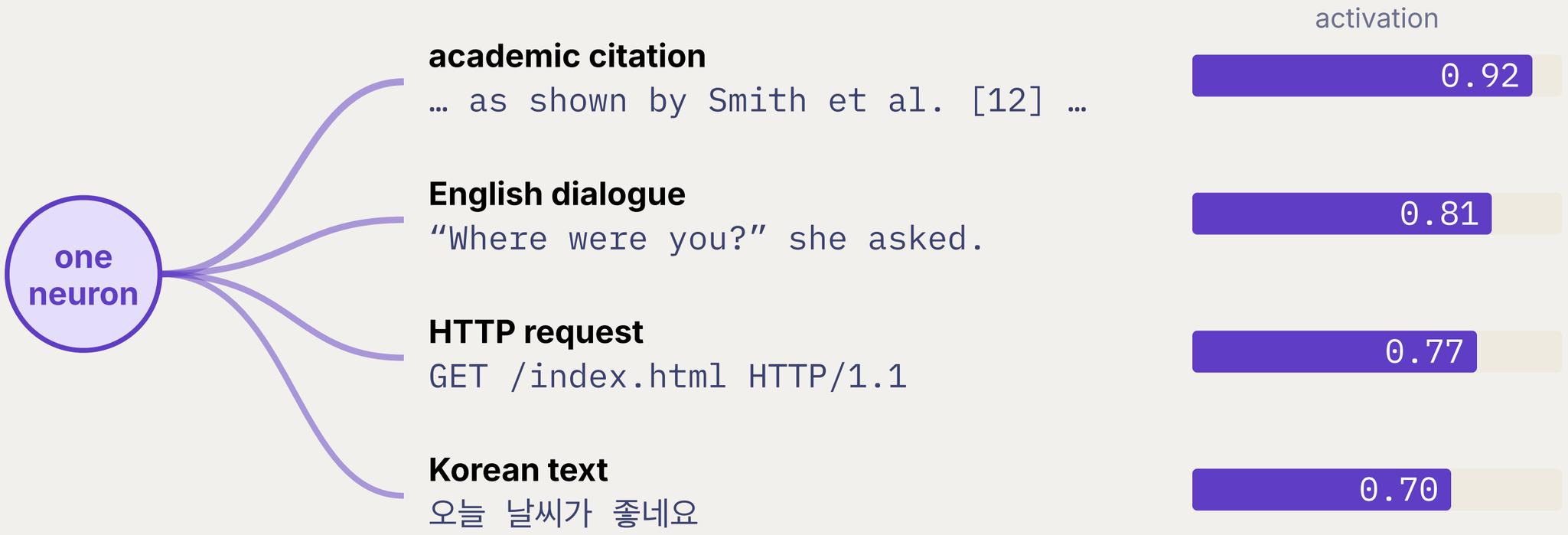
3

Decompose: what are the units?

Patching found where. Now we look for the variables inside an activation.

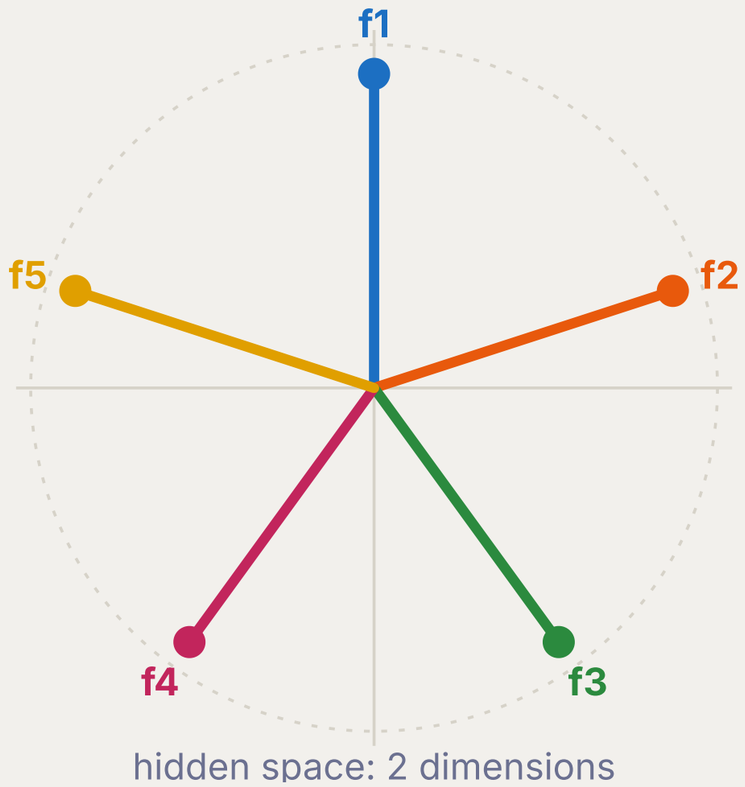
BLOCK 3 · DECOMPOSE

A single neuron often fires for unrelated things



Kinds of contexts after Bricken et al. 2023 (“Towards Monosemanticity”); activation values illustrative.

Sparse features can share dimensions



Sparse features: pack all 5

rarely on together; ReLU removes negative leaks

5 features in 2 dimensions

More features than dimensions,
paid for with interference.

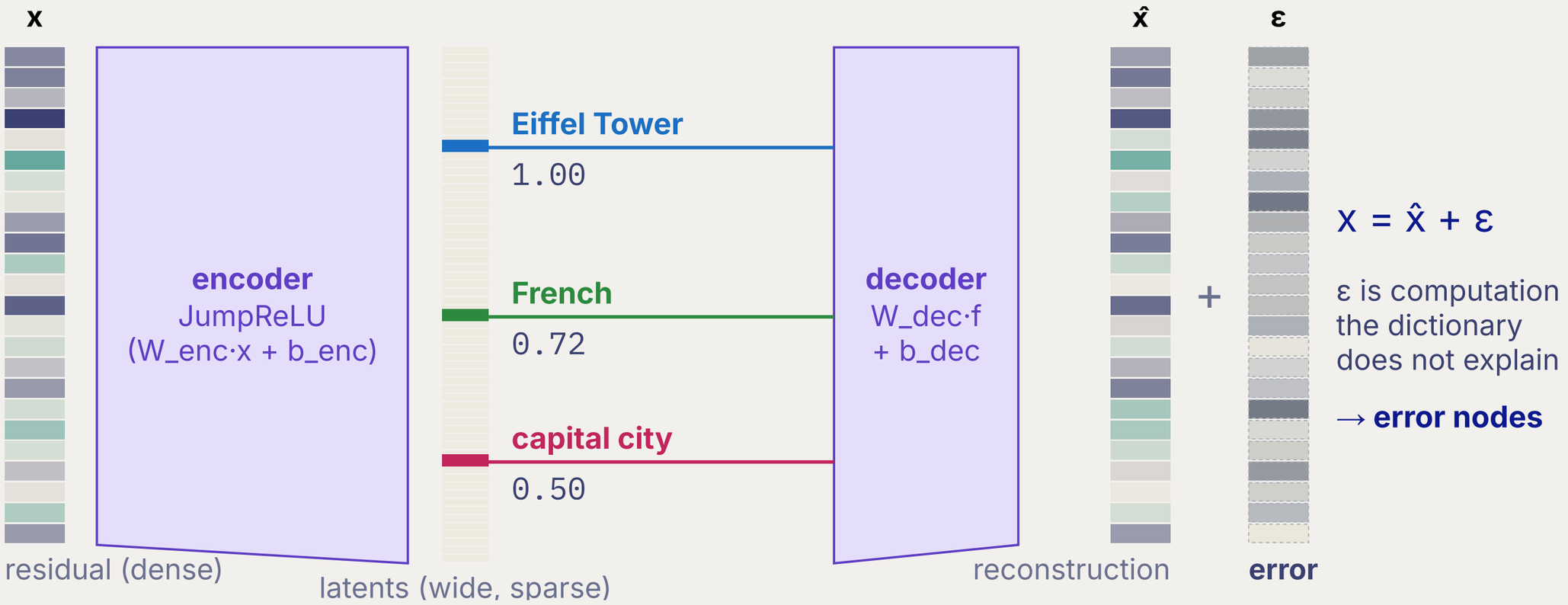
→ **a neuron mixes several features**

feature sparsity

dense — sparse

Illustrative toy geometry after Elhage et al. 2022, “Toy Models of Superposition”.

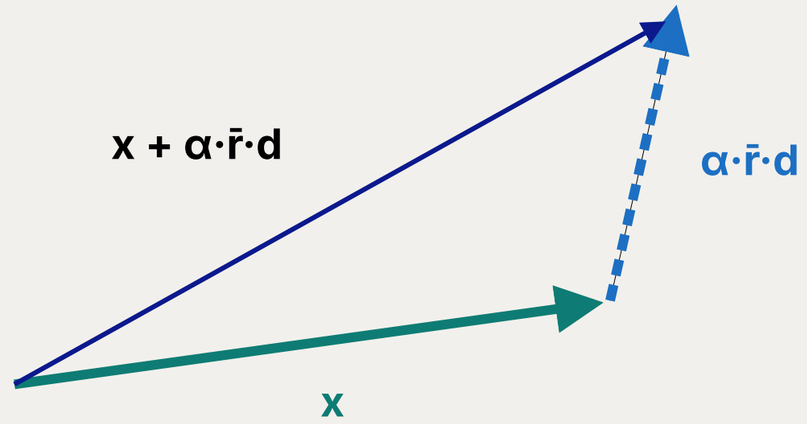
An SAE is a learned dictionary of directions



JumpReLU = a ReLU with a learned threshold per latent (what Gemma Scope 2 uses). Latent labels and values illustrative.

Gemma Scope 2 opens SAEs on every layer of Gemma 3

- SAEs and transcoders for every layer of Gemma 3, 270M to 27B.
- **Neuronpedia**: browse each feature's top activating texts.
- **Steering**: add a feature's decoder direction, scaled to a fraction of the residual norm; the output changes.



d: unit decoder direction · r̄: mean residual norm

```
from sae_lens import SAE
sae = SAE.from_pretrained("gemma-scope-2-1b-it-res", "layer_17_width_16k_l0_medium")
```

Google DeepMind, Gemma Scope 2 (2025). The SAE NB3 loads for Gemma 3 1B IT (SAELens 6.54 ids).

BLOCK 3 · DECOMPOSE

Before you run it: which probe wins, raw or SAE?

PREDICT

Will a linear probe trained on raw activations do better or worse than one trained on SAE latents?

Same labels, same layer, same 20 lines of scikit-learn.

Raw probe wins

Raw keeps all the information.

SAE probe wins

SAE latents are the "real" units.

About the same

Write down why.

NB3 · FEATURES & PROBES ≈ 14 min · Gemma 3 1B IT + Gemma Scope 2 · T4

Find a feature, steer with it, then race two probes

1. Load the layer-17 SAE; find a Mexico latent.
2. Check its label on Neuronpedia: is it really Mexico?
3. Steer at α = 0.25 of the residual norm; compare a random direction.
4. Race a raw probe against SAE probes, in and out of distribution.

Done when

You have IID and OOD accuracies for both probes.

No GPU?

The notebook switches to GPT-2 small and its SAE on CPU.

SAE probes are not reliably better than plain probes

113

datasets: SAE probes won on some, but not consistently, and did not beat strong baselines on average (Kantamneni et al., ICML 2025).

Mar 2025

Google DeepMind deprioritized fundamental SAE research after SAEs underperformed linear probes on out-of-distribution harmful-intent detection.

Your own race may go either way: one 200-sentence dataset is an anecdote. Benchmark against a plain probe before trusting a feature.

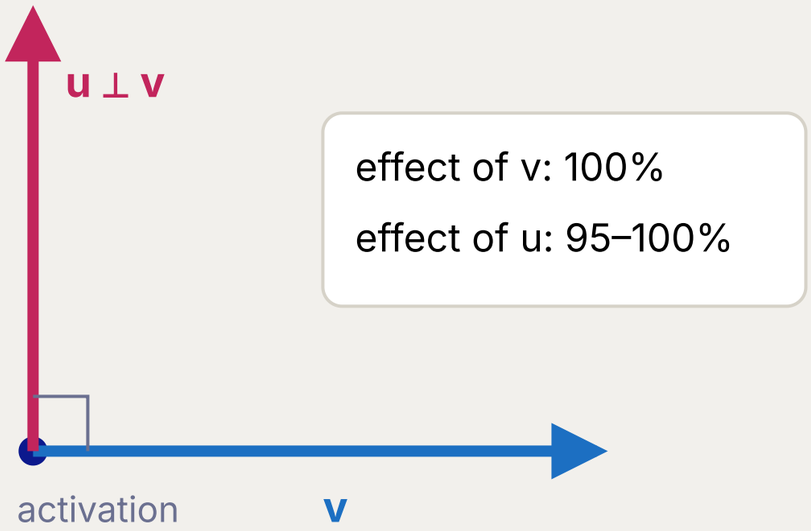
BLOCK 3 · DECOMPOSE

Probes detect what you name; SAEs find what you did not

	LINEAR PROBE	SAE
Needs	labels for a concept you chose	no labels
Good for	cheap, supervised detection	unsupervised discovery of units
Weak at	concepts nobody thought to label	reliably beating a probe on a known label

> Use them together: SAEs suggest hypotheses, probes and patching test them.

A steering effect does not identify the representation



In their experiments, directions orthogonal to a steering vector reached **95–100%** of its effect.

So "steering with feature f changes the output" shows f is *sufficient* to cause the change, not that f is *the* variable the model uses.

Venkatesh & Kurapath, “On the Non-Identifiability of Steering Vectors in LLMs”, ICLR 2026 workshop (Re-Align), arXiv 2602.06801.

1:40 · SALVADOR HINOJOSA

4

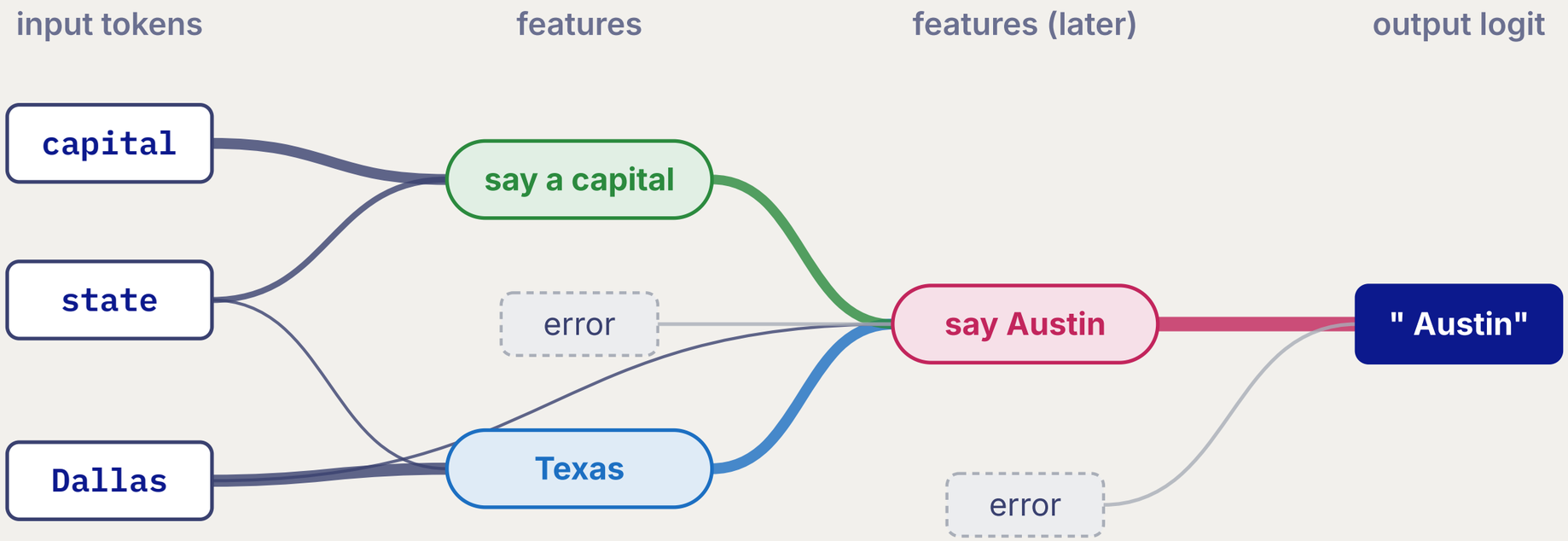
Trace: how does one answer get made?

Units connected into a graph for a single prompt, then tested by intervention.

BLOCK 4 · TRACE · ATTRIBUTION GRAPH

A two-hop answer passes through a "Texas" feature

"Fact: the capital of the state containing Dallas is" → **" Austin"**



Redrawn and simplified from Anthropic, “On the Biology of a Large Language Model” and “Circuit Tracing” (2025); edge weights illustrative. Open-sourced as circuit-tracer + Neuronpedia.

BLOCK 4 · TRACE · METHOD

A graph comes from a replacement model, one prompt at a time

1 · REPLACE

Swap the MLPs for per-layer or cross-layer transcoders: sparse, interpretable features. Keep an error node for what they miss.

2 · ATTRIBUTE

On one prompt, compute the direct linear effect between features, tokens, error nodes and logits.

3 · PRUNE

Keep the nodes and edges that carry most of the effect on the chosen logit.

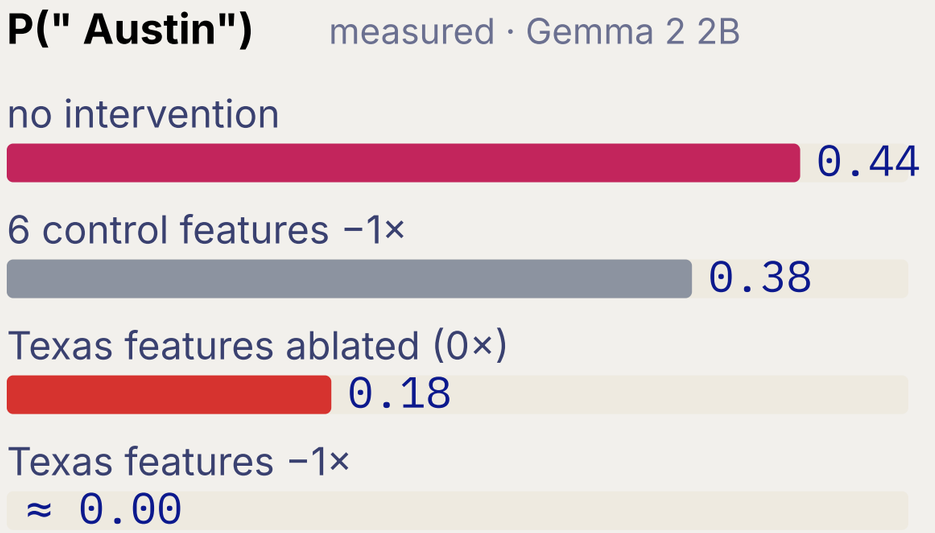
Attention patterns are frozen from the real run, so the effects are linear and exact for that prompt.

BLOCK 4 · TRACE · VALIDATION

A graph is a hypothesis until an intervention confirms it



steer features at the Dallas token, rerun, read the output



Texas −1× + California +0.5×
P(" Sacramento") = 0.135, tied for top

Measured in NB4 (Gemma 2 2B, featured Dallas graph, Neuronpedia steering). The same two tests appear in “On the Biology of a Large Language Model” (2025).

NB4 · CIRCUIT TRACING ≈ 14 min · Neuronpedia in the browser · no GPU needed

Trace your own two-hop prompt, then break the Dallas one on purpose

1. **+ New Graph** for your own prompt, e.g. Ciudad Juárez → Chihuahua.
2. On the featured Dallas graph, find Texas (e.g. layer 16 #25).
3. Pin it, **Steer** it to −1×, and watch Austin drop.

Done when

One intervention changed the answer the way the graph predicts.

Opt-in

Own GPU runtime for NB4? Build the graph with circuit-tracer.

BLOCK 4 · TRACE · LIMITS

Attribution graphs explain one prompt, partially

Local

One prompt. A different phrasing can use a different path.

Incomplete

Error nodes carry computation nobody has explained.

Attention partly hidden

Patterns are frozen, so why a head attends where it does is not in the graph.

Needs validation

Feature labels are interpretations. Test them by intervention.

BLOCKS 2 – 4 · RECAP

Each method answers one question and leaves one open

METHOD	ANSWERS	LEAVES OPEN
Patching	where the model relies on information	what that information is
SAEs · probes · steering	which units are present; can we detect them	whether they are *the* variables
Attribution graphs	how units combine on one prompt	whether it generalizes

> The common rule: an explanation counts when an intervention can test it.

HANDOFF · 2:10

A plain probe can match an SAE feature, and patching tests what a model relies on.

Next: Víctor Terrón-Macías · Block 5 · Audit

2:10 – 2:50 · VÍCTOR TERRÓN-MACÍAS

5

Audit: can we trust the judge?

Mechanistic interpretability as a confidence proxy for LLM-assisted audits.

BLOCK 5 · AUDIT

Two tools you already have, one new subject: an LLM judge

> *“A plain probe can match an SAE feature, and patching tests what a model relies on.”*

Linear probe
reads what the hidden state represents

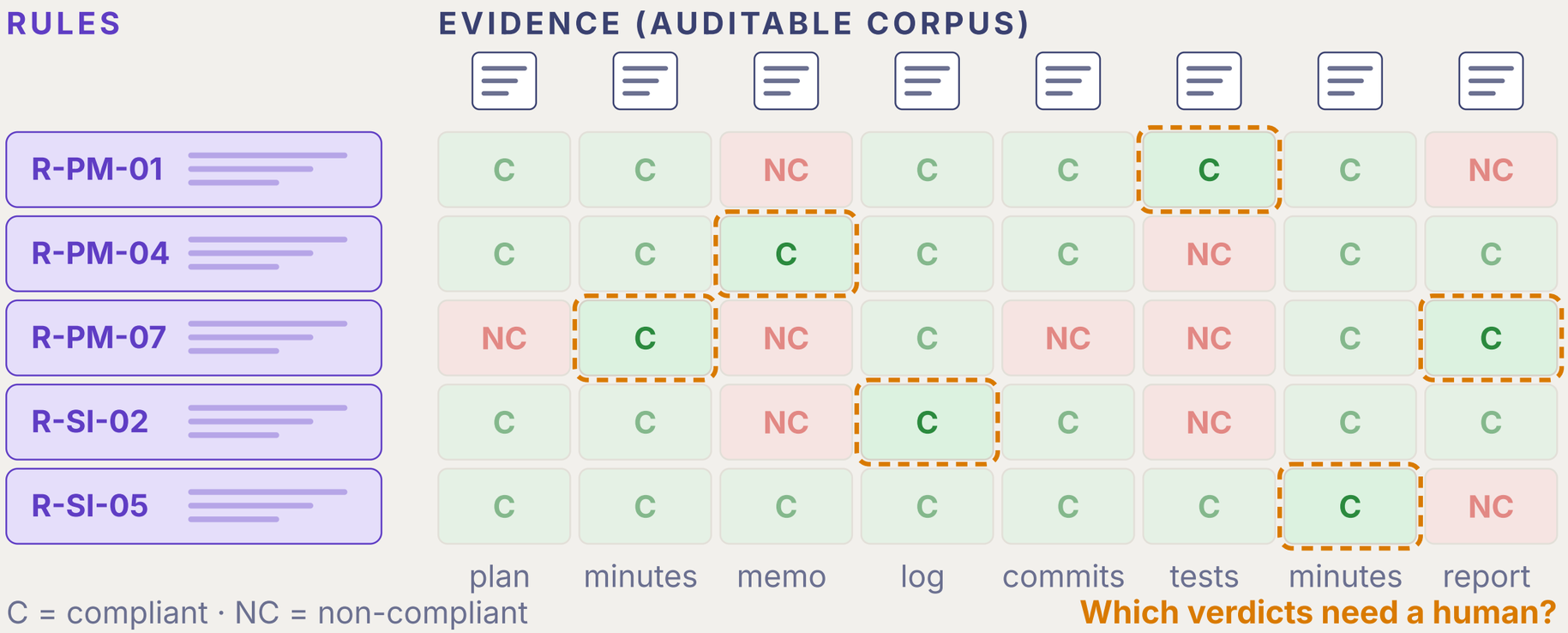
→ **Is this verdict likely wrong?**
a confidence signal read from inside, not from what the judge says

Intervention
here: editing the evidence text

→ **Does it rest on the right evidence?**
a grounding check for one verdict

BLOCK 5 · AUDIT

An LLM-assisted audit is a grid: every rule against every piece of evidence



Illustrative sizes. A real audit: tens of rules × hundreds of documents = thousands of verdicts.

BLOCK 5 · AUDIT · CONTEXT

Our motivating case: process audits of very small software teams

ISO/IEC 29110

Lifecycle profiles for **very small entities**: teams of up to 25 people. Covers project management and software implementation.

TR 29110-3-1

Guidelines for **assessing** those processes: check each expected practice against the project's own records.

> An assessor reads plans, minutes and change logs, rule by rule. That is the grid an LLM judge could help fill.

Paraphrased. The standard's text is licensed, so we never quote it; the rule on the next slides is invented.

Invented example, in the style of ISO/IEC 29110 · numbers illustrative

One rule, one piece of evidence: is the project compliant?

RULE R-PM-07

Every change request is 1 recorded, its 2 impact on schedule and cost is assessed, and 3 the customer approves it before work on it starts.

EVIDENCE · MEETING MINUTES

“12 Mar status meeting. Client asked to add CSV export to the reports module. Dev lead estimates 3 extra days; added to Sprint 4. Action: update the project plan.”

Invented example · numbers illustrative

The judge is confident — and wrong

LLM JUDGE

Compliant

“The request was recorded and its schedule impact assessed.”

Verbalized confidence 90%

HUMAN AUDITOR (GOLD)

Non-compliant

met	1 · recorded
missing	2 · cost impact not assessed
missing	3 · no customer approval before scheduling

90% confident · wrong

Invented example · numbers illustrative

A probe on the judge's residual stream disagrees, so a human gets the case



Invented example · numbers illustrative

Editing the evidence shows the verdict rests on one clue and ignores another

EVIDENCE · MEETING MINUTES

12 Mar status meeting. Client asked to add CSV export to the reports module. Dev lead estimates 3 extra days; added to Sprint 4. Action: update the project plan. **The customer rejected the change.**

JUDGE VERDICT

Compliant

Remove the schedule sentence	flips
Add a customer rejection	no change

Edit 2: schedule restored, rejection added

> The judge relies on the schedule clue and ignores the approval clause.

In one study, a judge's hidden states predicted its errors better than its words

Better calibrated

Brier-trained linear probes beat verbalized and multi-sample confidence, at about 10× less compute.

Transfers — there

On their reasoning judges, it held up on unseen domains. You will test that on ours.

Conservative

Under-confident on easy sets — a fair price when a missed error costs more than a review.

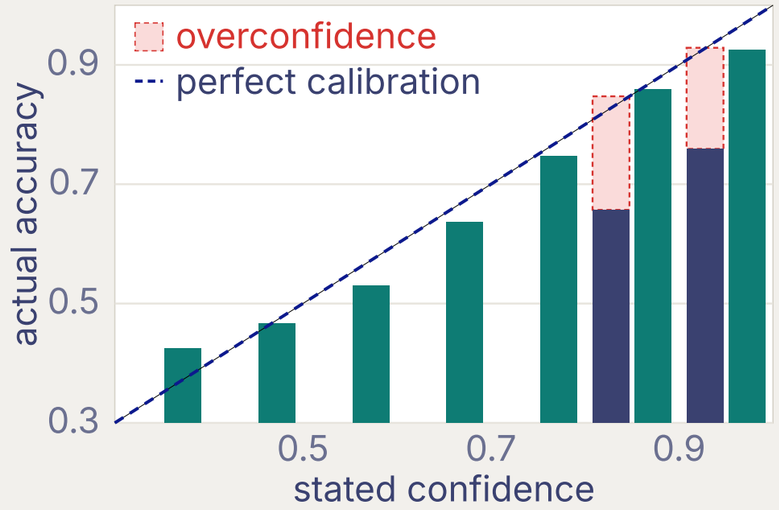
> LLM judges are overconfident in words. The cheap baseline every probe must beat: the judge's own Yes/No token probability.

Brier score = mean squared gap between confidence and the 0/1 outcome (lower is better). Radharapu et al., “Calibrating LLM Judges: Linear Probes for Fast and Reliable Uncertainty Estimation”, ACL 2026 Industry, arXiv 2512.22245. Related: semantic entropy probes, Kossen, Han et al. 2024, arXiv 2406.15927.

Synthetic illustration · not measured

A calibrated score turns review into a dial

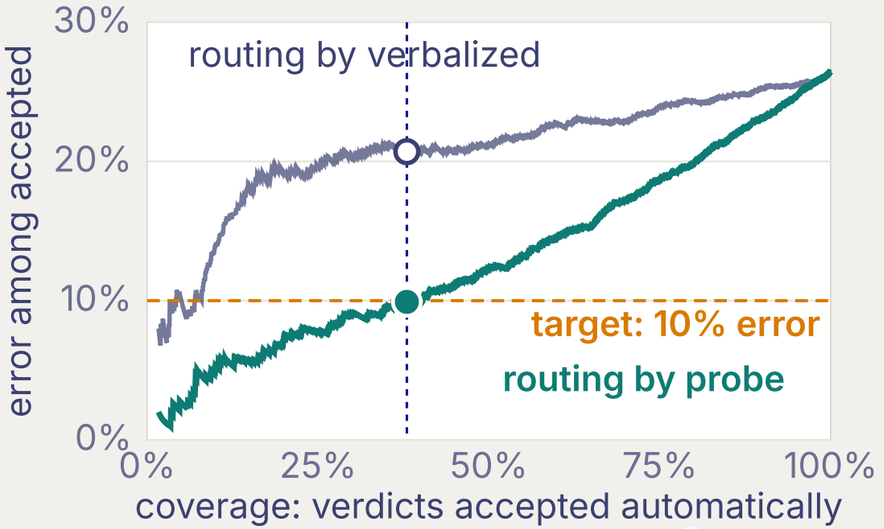
RELIABILITY DIAGRAM



Verbalized confidence · **ECE 0.17**

Probe on hidden state · **ECE 0.01**

RISK–COVERAGE



τ = 0.83

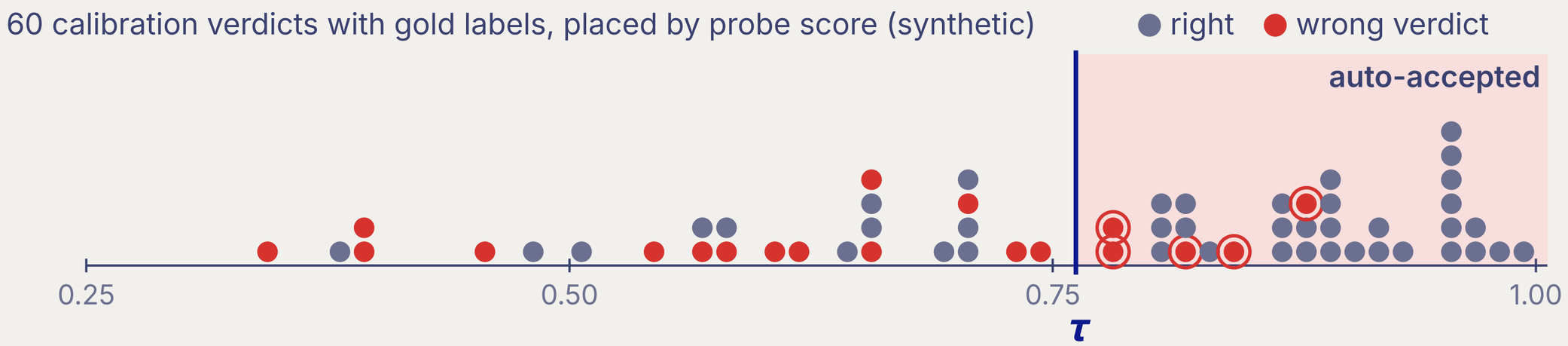
review threshold τ on probe score · drag or press →

auto-accept 38% · **62% to a human** · **3.8% escape**

Synthetic: 3,000 simulated verdicts. ECE = average gap between stated confidence and actual accuracy. Escape = wrong and auto-accepted, over all verdicts.

Research direction · not a published result

A conformal threshold could bound how many wrong verdicts escape review



α = 10% · **5 of 60 escape** (wrong and ≥ τ) · (60/61)·(5/60) + 1/61 = 9.8% ≤ 10% · auto-accept 34 of 60

Lowest τ with (n/(n+1))·escape(τ) + 1/(n+1) ≤ α on calibration data
→ expected escape rate ≤ α.

Holds only if new cases look like calibration cases. A new process breaks that: test it.

BLOCK 5 · AUDIT · HANDS-ON STAND-IN

In the notebook, contracts stand in for project records

AUDIT (R-PM-07)	LEGALBENCH CONTRACT-NLI
Rule	Assertion — one of 14 contract-NLI tasks
Evidence	Contract excerpt from an NDA
Verdict	Does this clause state that … ? Yes / No
A new 29110 process	3 held-out assertions (distribution shift)
Judge	Gemma 3 4B IT · cached hidden states, no GPU needed

LegalBench contract_nli_* tasks (CC BY 4.0): Guha et al., NeurIPS 2023 (Datasets & Benchmarks); derived from ContractNLI, Koreeda & Manning 2021.

BLOCK 5 · AUDIT · BEFORE YOU RUN

Commit to a prediction first

PREDICT Will the probe beat the judge's own confidence at flagging its wrong verdicts — and will it still work on assertions it never saw?

“The judge's own confidence” = two signals: what it says (0–100) and its Yes/No token probability.

A

Beats both, and transfers

B

Beats both, not on unseen assertions

C

Beats words, only ties token probability

D

Neither: token probability wins

BLOCK 5 · AUDIT · HANDS-ON

NB5: measure whether the judge's hidden state knows when it is wrong

1. Run the judge on a few pairs
2. Train a probe to predict wrong verdicts
3. Compare with verbalized and token probability: AUROC, ECE
4. Pick a threshold: ≤ 10% error among accepted
5. Test on 3 held-out assertions
6. Remove the evidence behind confident verdicts

22

minutes, in pairs

NB5 judge confidence
Colab · CPU is fine
(cached activations)

AUROC	chance a wrong verdict scores lower than a right one (0.5 = useless)
ECE	average gap between stated confidence and actual accuracy
Brier	mean squared gap between confidence and the 0/1 outcome

BLOCK 5 · AUDIT · DEBRIEF

What would an auditor accept as evidence that a verdict is safe to skip?

Where did the probe win?

Against words, and against token probability?

Where did it fail?

Held-out assertions. Ungrounded verdicts that are still correct.

What counts as evidence?

A score, a bound, a grounding check — or all three?

BLOCK 5 · AUDIT · METHOD MAP

Pick the method from your question, and the validation from the method

YOUR RESEARCH QUESTION	METHOD	VALIDATE WITH
Is concept X represented?	**Linear probe**	Held-out and OOD splits — BLOCK 5
Which component causes Y?	**Activation patching**	Other corruptions, path patching — BLOCK 5 · TEXT
Which features does it use?	**SAE / transcoder**	Beat a plain probe baseline
How is one prompt computed?	**Attribution graph**	Intervene on graph features
Can I control behaviour Z?	**Steering vector**	Try orthogonal directions

BLOCK 5 · AUDIT · YOUR TURN

Your turn: map one LLM-judged task from your own research

1
LLM-judged task
What does the model decide?

2
Ground truth
Who or what gives the gold label?

3
Confidence signal
Probe on which layer, which token?

4
Threshold policy
What escape rate can you tolerate?

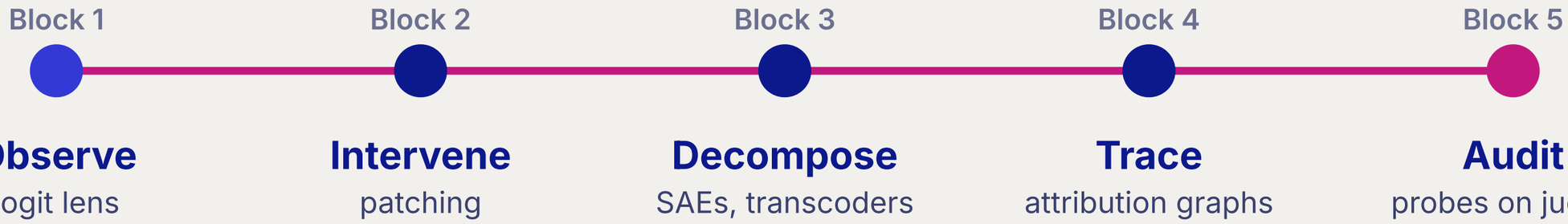
5
Grounding check
Which evidence should flip the verdict?

6
Shift test
What is your “unseen assertion”?

2 minutes in pairs · printed transfer canvas on your table

WRAP · WHERE THE FIELD IS GOING

From observing models to auditing them — with tools that are still young



Open problems

- Global circuits, not one prompt at a time
- Are replacement models faithful?
- Agents that do interpretability
- Shared evaluation standards

Implications

- Assurance and audits of AI systems
- Evidence for regulation
- Scientific discovery inside models

WRAP · RESOURCES

Everything we used today is open

TOOLS AND COURSES

TransformerLens
transformerlensorg.github.io/TransformerLens

SAELens · Gemma Scope 2
decoderesearch.github.io/SAELens

nnsight · circuit-tracer
nnsight.net · pypi.org/project/circuit-tracer

Neuronpedia
neuronpedia.org

Transformer Explainer
poloclub.github.io/transformer-explainer

ARENA · chapter 1
learn.arena.education

LegalBench · Guha et al. 2023
hazyresearch.stanford.edu/legalbench

PAPERS BEHIND TODAY

Interpretability in the Wild (IOI) · Wang et al. 2022
arxiv.org/abs/2211.00593

Toy Models of Superposition · Monosemanticity
transformer-circuits.pub (2022, 2023)

Circuit Tracing · Biology of an LLM
transformer-circuits.pub/2025/attribution-graphs

Are SAEs Useful? · Kantamneni et al. 2025
arxiv.org/abs/2502.16681

Open Problems · Sharkey et al., TMLR 2025
arxiv.org/abs/2501.16496

Calibrating LLM Judges · Radharapu et al. 2026
arxiv.org/abs/2512.22245

Gemma Scope 2 docs
ai.google.dev/gemma/docs/gemma_scope

Thank you

Miguel González
Tecnológico de Monterrey

Salvador Hinojosa
Tecnológico de Monterrey

Víctor Terrón-Macías
Tecnológico de Monterrey

Slides, notebooks and printables:

salvahin.github.io/mechinterp-tutorial